

Introduction to Garbage Collection

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

The runtimes of many programming languages feature **dynamic memory management**.

The first aspect of that management is **allocation**, that is, the program can create pieces of data (called in this context **objects**) whose lifetime can be that of the program itself. In particular, they can outlive the procedure or function that created them.

Those objects exist in a memory area called the **heap**. More precisely, a **continuous block** is allocated for one object, whose size depends on the **type** of the object.

Garbage Collection

For example, in Java, we can declare a type for the cells in a linked list of integers as follows:

```
class Cell {  
    int    content;  
    Cell  next;  
}
```

Each cell contains an integer and a **pointer** to another cell. The pointer is implicit in the recursive occurrence of the type name `Cell`. It is either the address of an object in memory, or a special value **nil** (noted `null` in Java). Consequently, each cell requires at least two **words** (one for each field and perhaps more, depending on the data layout).

Garbage Collection

A cell can be allocated with the built-in function `new`, like so:

```
Cell c = new Cell ();
```

A word is the smallest amount of memory that can be addressed and filled by a primitive type or a pointer — 64 bits in modern computers.

In other words, we can imagine memory as a table with addresses as keys and words as values. Those values may be interpreted as another key (address) or not (primitive type).

Then we can modify the contents of the fields it contains, as many times as we wish to. Every call to `new` returns the **address of a new block** drawn from the available memory.

Garbage Collection

Because each allocation removes from available, finite memory, it is necessary for the GC to **free the memory used by useless objects**, which it needs to define and find.

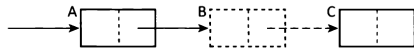
In some programming languages, like C, this task is entirely delegated to the programmers. They have a predefined procedure `free` that frees a memory block:

```
free(c);
```

After it is executed, the block pointed at by `c` is considered available again, so its address can be reused in a future allocation.

Garbage Collection

In C, the premature of an object may lead to errors:



Here, B has been freed. The live object A now contains a **dangling pointer**, and the memory occupied by C has **leaked**: C is not reachable, but it cannot be freed.

Another common mistake is freeing an already freed object (like B above): this is an undefined behaviour and thus a source of **non-determinism**.

Garbage Collection

In Java, there is no equivalent to the `free` procedure of C because the identifying and freeing of useless objects is left to the runtime system.

That part of the runtime is called the **garbage collector** (GC). At regular intervals, it interrupts the program, analyses the heap, find out the useless objects and free them.

This technique was invented by McCarthy, creator of the functional programming language List in 1960. It is widely used in mainstream languages like Java, C#, Haskell or OCaml.

Beyond avoiding the two errors above, a GC may also free many objects at a time, which is more efficient than any sequential, manual deallocation.

Garbage Collection

How does a GC determine what objects are useless, since it has no access to the source code or the programmer (interactively)?

The GC can rely on the following condition:

If, at a certain moment, a program does not have a pointer (anymore) to an object, that object is considered useless.

That condition is not true in C, where pointers can be forged (type casting) from integers, but it holds in Java for example.

Therefore, the memory block of that object can be reused for another, without the program being able to observe that reuse.

Garbage Collection

When a program has the address of an object, it is said to be **alive**, and **dead** otherwise.

We can define inductively the notion of liveness:

Definition 1

*Both the global and local variables of the program are called **roots**. By definition (base case), the object pointed at by a root is alive.*

Moreover, if an object is alive and one of its fields contains the address of another object, then that latter object is alive too (inductive case).

What counts as global and local variable depends on the programming language. In Java, the former are the `static` fields (a.k.a. instance fields), and the latter are methods arguments (including `this`) and the variables declared in the method being executed.

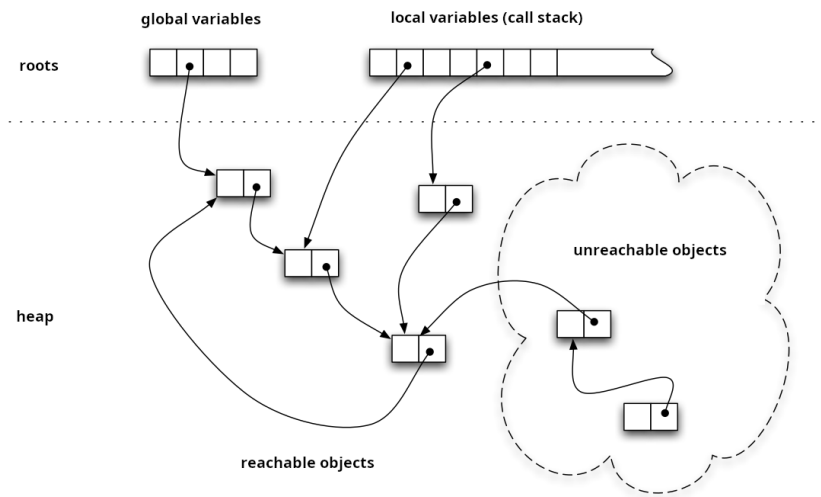
Garbage Collection

The heap can now be seen as a **graph**, where the **vertices** correspond to the roots and the objects, and the **edges** model the pointers between objects (that is, if one's block contains a pointer to another's).

The live objects are then the vertices **reachable from the roots**.

Therefore, to determine the live objects at a given time, the GC has to traverse the graph of objects from the roots.

Garbage Collection



Garbage Collection

We started this introduction by referring to 'useful objects', but now we see that the appropriate concept is that of 'live objects', which is the same as 'reachable objects'.

It is obvious that all dead objects are useless, but the reciprocal is not true: a live object might be useless if the program does nothing with it. They are allocated in the heap but cannot be reclaimed. This is a case of **memory leak**. (We saw page 6 another case of memory leak in C.)